Forward and Reverse Mode AD

Jacobian Computation in `Expression_Differentiator`

A method-by-method walkthrough with worked examples

Expression_Differentiator — internal documentation

Contents

1 Introduction **2**

2 Expression Tree Representation **2**

2.1 Node types . . . 2

2.2 Tree for the running example . . . 2

2.3 Universal methods on every node . . . 3

2.4 `eval()` . . . 3

2.5 `update(symbols, updates)` . . . 3

2.6 `collect(symbols, out)` . . . 4

2.7 `eval_seeded<Syms>(vals)` . . . 4

2.8 `backward(syms, adj, grads)` . . . 5

2.9 Operation-level `backward` rules . . . 5

3 Dual Numbers — the Engine of Forward Mode **6**

3.1 Definition . . . 6

3.2 Implementation: `Dual<T>` (`dual.hpp`) . . . 7

3.3 Component access . . . 7

4 Forward-Mode Jacobian **8**

4.1 Mathematical basis . . . 8

4.2 Code walkthrough: `detail::forward_mode_gradient` . . . 8

4.3 Flowchart . . . 9

4.4 Worked example . . . 9

5 Reverse-Mode Jacobian **10**

5.1 Mathematical basis . . . 10

5.2 Code walkthrough: `detail::reverse_mode_gradient` . . . 10

5.3 Flowchart . . . 11

5.4 Worked example . . . 11

6 Comparison **12**

6.1 Public API (`gradient.hpp:196-211`) . . . 12

7 Symbol Resolution: Compile-Time Indexing **13**

8 Quick Reference **13**

1 Introduction

This document explains how the library computes Jacobians (first-order derivatives) using *forward-mode* and *reverse-mode* automatic differentiation (AD). Every concept is grounded in the actual C++ types, with line references to the source.

Running example. Throughout, we use the scalar function of two variables

$$f(x,\, y) \;=\; x\,y \;+\; \sin(x), \qquad (x, y) = \left(\tfrac{\pi}{2},\; 3\right).$$

Its Jacobian (gradient row-vector) is

$$\nabla f = \begin{pmatrix} \dfrac{\partial f}{\partial x} & \dfrac{\partial f}{\partial y} \end{pmatrix} = \begin{pmatrix} y + \cos x & x \end{pmatrix} \Big|_{x=\pi/2,\, y=3} = \begin{pmatrix} 3 + 0 & \frac{\pi}{2} \end{pmatrix} = \begin{pmatrix} 3 & \frac{\pi}{2} \end{pmatrix}.$$

2 Expression Tree Representation

2.1 Node types

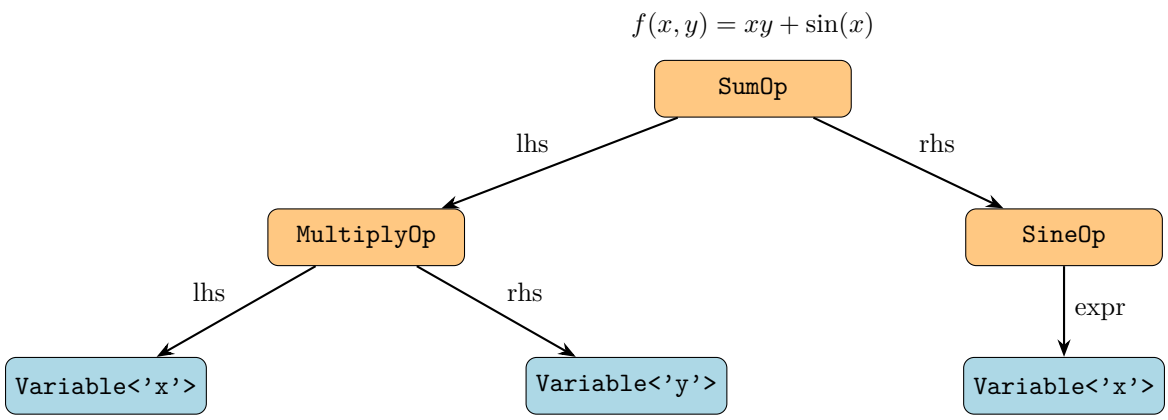
Every expression is an immutable tree of nodes. There is no virtual dispatch; nodes communicate through C++20 concepts.

Node type	Template signature	Role
`Constant<T>`	`Numeric T`	Fixed numeric literal
`Variable<T,symbol>`	`Numeric T`, `char symbol`	Named mutable scalar
`RuntimeVariable<T>`	`Numeric T`	Index-addressed variable
`Expression<Op,LHS,RHS>`	binary op, two child nodes	Binary operation node
`MonoExpression<Op,Expr>`	unary op, one child node	Unary operation node
`EvalResult<T>`	`Numeric T`	Thin value wrapper

Table 1: First-class expression node types (`expressions.hpp`, `values.hpp`).

The concept `ExpressionConcept` is satisfied by all rows above via the `is_expression_type` tag trait (`expressions.hpp:38-67`).

2.2 Tree for the running example



The symbol `'x'` appears in two leaves. Both have type `Variable<T,'x'>` and are updated in sync by `update()`.

2.3 Universal methods on every node

Every node unconditionally provides these five methods:

Method overview

```cpp
// 1. Evaluate: returns scalar at current variable state.
auto eval() const -> value_type;

// 2. Walk tree and write new values into every Variable.
void update(const auto& symbols, const auto& updates);

// 3. Walk tree and read current Variable values into an array.
void collect(const auto& symbols, auto& out) const;

// 4. Stateless evaluation from an explicit seed array (no mutation).
template<typename Syms, size_t N>
auto eval_seeded(const array<value_type,N>& vals) const -> value_type;

// 5. Reverse-mode adjoint propagation.
void backward(const auto& syms, value_type adj, auto& grads) const;
```

2.4 `eval()`

Purpose. Recursively evaluate the expression tree, returning the scalar primal value.

Per-node behaviour:

- `Constant`: returns stored literal (`values.hpp:176`).
- `Variable<T,C>`: returns internal `value` (`values.hpp:218`).
- `RuntimeVariable`: returns `value_` (`values.hpp:311`).
- `Expression<Op,L,R>`: calls `Op::eval(lhs, rhs)` via `std::apply`; delegates to the C++ operator (e.g. `std::plus<T>{}`) (`expressions.hpp:162`).
- `MonoExpression<Op,E>`: calls `Op::eval(expression)` (`expressions.hpp:107`).

Example: $f(\pi/2, 3)$

$$\texttt{root.eval()} = \underbrace{x \cdot y}_{\texttt{mul.eval()}} + \underbrace{\sin(x)}_{\texttt{sine.eval()}} = \frac{\pi}{2} \cdot 3 + 1 = \frac{3\pi}{2} + 1 \approx 5.712$$

2.5 `update(symbols, updates)`

Purpose. Walk the tree and write new values into every `Variable` node belonging to the symbol list `symbols`.

`symbols` is a Boost.MP11 compile-time type-list `mp_list<integral_constant<char,C>,...>`, one entry per unique variable. `updates` is an array of `value_type` aligned with that list.

Per-node behaviour:

- `Constant`: no-op (`values.hpp:184`).

- `Variable<T,symbol>`: resolves `constexpr idx = find_index_of_char<symbol,Syms>()`, then `*this = updates[idx]` (`values.hpp:265-270`).
- `RuntimeVariable`: `value_ = T(updates[index_])` (`values.hpp:317-319`).
- `Expression`: recurses into both children (`expressions.hpp:182-185`).
- `MonoExpression`: recurses into its child (`expressions.hpp:116-118`).

Key invariant

After `expr.update(symbols{}, seeds)`, every `Variable` in the entire tree has been refreshed. Calling `expr.eval()` afterward reflects the new values. This is the mechanism by which forward-mode seeds `Dual` numbers into the tree before each directional-derivative pass.

2.6 `collect(symbols, out)`

Purpose. The inverse of `update`: snapshot current variable values *from* the tree into `out`.

Used by the no-argument `hessian()` overloads to capture the evaluation point before reseeding.

- `Constant`: no-op.
- `Variable<T,symbol>`: `out[idx] = value` (`values.hpp:272-278`).
- `RuntimeVariable`: no-op (`values.hpp:320`).
- `Expression`/`MonoExpression`: recurses.

2.7 `eval_seeded<Syms>(vals)`

Purpose. A *stateless* evaluation that bypasses internal `Variable` fields entirely. Every `Variable` node reads its value from the caller-supplied array `vals` instead.

Per-node behaviour:

- `Constant`: returns literal, ignores `vals` (`values.hpp:189`).
- `Variable<T,symbol>`: `return vals[find_index_of_char<symbol,Syms>()]` (`values.hpp:232-235`).
- `RuntimeVariable`: returns `value_`, ignores `vals` (`values.hpp:327`).
- `Expression<Op,L,R>`: recursively calls `eval_seeded` on both children, wraps each result in `EvalResult`, calls `Op::eval` (`expressions.hpp:172-180`).
- `MonoExpression<Op,E>`: same pattern with one child (`expressions.hpp:109-114`).

Why EvalResult?

`Op::eval` requires its arguments to satisfy `ExpressionConcept`. A bare scalar `T` does not. `EvalResult<T>` is a zero-overhead wrapper that does:

```
template <Numeric T> struct EvalResult {  // expressions.hpp:59-64
  T value;
  constexpr T eval() const { return value; }
  constexpr operator T() const { return value; }
};
```

It satisfies `ExpressionConcept` yet carries only a scalar, allowing `eval_seeded` to reuse the exact same `Op::eval` path as `eval()` with no code duplication. The compiler eliminates it entirely.

`eval_seeded` is the workhorse of the *forward-mode Hessian*, where the expression is `const` and must be evaluated at many different `Dual<Dual<T>>` seed arrays without mutating it.

2.8 `backward(syms, adj, grads)`

Purpose. Reverse-mode adjoint propagation. Given that this node's output is weighted by the upstream adjoint `adj` ($= \partial f/\partial v$), accumulate each variable's contribution into `grads`.

Parameters:

- `syms` — compile-time symbol list.
- `adj` — adjoint flowing *into* this node: how much the final output f depends on this node's output.
- `grads` — output array; slot i accumulates $\partial f/\partial x_i$ via **additive** `+=`.

Per-node behaviour:

Constant. No variable, no contribution.

```
constexpr void backward(const auto&, T, auto&) const {}  // values.hpp
    :186
```

Variable⟨T,symbol⟩. This is a leaf: $\partial(\text{self})/\partial x_{\text{self}} = 1$, so accumulate the full adjoint.

```
// values.hpp:286-292
constexpr void backward(const auto& syms, T adj, auto& grads) const {
    constexpr auto idx = find_index_of_char<symbol, Syms>();
    grads[idx] += adj;
}
```

RuntimeVariable. `grads[index_] += adj` (`values.hpp:322-324`).

Expression⟨Op,L,R⟩. Delegates to `Op::backward(lhs, rhs, adj, syms, grads)` (`expressions.hpp:190`

MonoExpression⟨Op,E⟩. Delegates to `Op::backward(expression, adj, syms, grads)` (`expressions.h`

2.9 Operation-level `backward` rules

Each `Op` struct's `backward` method encodes the chain rule for one mathematical operation.

SumOp: $f = u + v$

$\dfrac{\partial f}{\partial u} = 1$, $\dfrac{\partial f}{\partial v} = 1$ — both children get the adjoint unchanged.

```
lhs.backward(syms, adj, grads);  // operations.hpp:65
rhs.backward(syms, adj, grads);  // operations.hpp:66
```

MultiplyOp: $f = u \cdot v$

$\frac{\partial f}{\partial u} = v$, $\frac{\partial f}{\partial v} = u$ — each child is scaled by the other operand's value.

```
lhs.backward(syms, adj * static_cast<T>(rhs), grads);  // operations.hpp:81
rhs.backward(syms, adj * static_cast<T>(lhs), grads);  // operations.hpp:82
```

DivideOp: $f = u/v$

$\frac{\partial f}{\partial u} = 1/v$, $\frac{\partial f}{\partial v} = -u/v^2$:

```
const T b = static_cast<T>(rhs);              // operations.hpp:108-114
lhs.backward(syms, adj / b, grads);
rhs.backward(syms, -adj * static_cast<T>(lhs) / (b * b), grads);
```

NegateOp: $f = -u$

```
expr.backward(syms, -adj, grads);  // df/du = -1
```

SineOp: $f = \sin(u)$

```
expr.backward(syms, adj * cos(static_cast<T>(expr)), grads); // df/du = cos(u)
```

ExpOp: $f = e^u$

```
expr.backward(syms, adj * exp(static_cast<T>(expr)), grads); // df/du = e^u
```

All other transcendentals follow the same pattern.

3 Dual Numbers — the Engine of Forward Mode

3.1 Definition

A *dual number* is a pair $(v, \dot{v})$ where v is the primal value and $\dot{v}$ is the tangent. Arithmetic extends ordinary arithmetic by $\varepsilon^2 = 0$:

$$(v, \dot{v}) + (u, \dot{u}) = (v + u,\ \dot{v} + \dot{u})$$

$$(v, \dot{v}) \cdot (u, \dot{u}) = (vu,\ \dot{v}u + v\dot{u}) \qquad \text{(product rule)}$$

$$\frac{(v, \dot{v})}{(u, \dot{u})} = \left(\frac{v}{u},\ \frac{\dot{v}u - v\dot{u}}{u^2}\right) \qquad \text{(quotient rule)}$$

Op	$f(u)$	local $\partial f/\partial u$	multiplier on `adj`
`NegateOp`	$-u$	-1	$-$`adj`
`SineOp`	$\sin u$	$\cos u$	`adj*cos(u)`
`CosineOp`	$\cos u$	$-\sin u$	`-adj*sin(u)`
`ExpOp`	e^u	e^u	`adj*exp(u)`
`TanOp`	$\tan u$	$\sec^2 u$	`adj/(cos(u)^2)`
`LogOp`	$\ln u$	$1/u$	`adj/u`
`SqrtOp`	$\sqrt{u}$	$1/(2\sqrt{u})$	`adj/(2*sqrt(u))`
`AbsOp`	$\lvert u\rvert$	$\mathrm{sign}(u)$	`adj*sign(u)`
`SinhOp`	$\sinh u$	$\cosh u$	`adj*cosh(u)`
`CoshOp`	$\cosh u$	$\sinh u$	`adj*sinh(u)`
`TanhOp`	$\tanh u$	$\mathrm{sech}^2 u$	`adj/(cosh(u)^2)`
`AsinOp`	$\arcsin u$	$1/\sqrt{1-u^2}$	`adj/sqrt(1-u*u)`
`AcosOp`	$\arccos u$	$-1/\sqrt{1-u^2}$	`-adj/sqrt(1-u*u)`
`AtanOp`	$\arctan u$	$1/(1+u^2)$	`adj/(1+u*u)`

Table 2: All unary `backward` rules (`operations.hpp`).

3.2 Implementation: `Dual<T>` (`dual.hpp`)

```cpp
template <typename T> class Dual {
    T val{};     // primal
    T deriv{};   // tangent

    constexpr Dual operator*(const Dual& o) const {     // dual.hpp:26-28
        return Dual{val*o.val,  deriv*o.val + val*o.deriv};  // product rule
    }
};

// Transcendentals apply the chain rule automatically:
friend constexpr Dual sin(const Dual& d) {              // dual.hpp:68-71
    return Dual{sin(d.val),  cos(d.val) * d.deriv};
}
friend constexpr Dual exp(const Dual& d) {              // dual.hpp:77-79
    const T e = exp(d.val);
    return Dual{e,  e * d.deriv};
}
```

`Dual<T>` satisfies the `Numeric` concept (`dual.hpp:127`), so `Variable<Dual<double>,'x'>` is a perfectly ordinary expression node whose value happens to carry a tangent.

3.3 Component access

`Dual<T>::get<I>()` returns `val` ($I = 0$) or `deriv` ($I = 1$). For nested `Dual<Dual<T>>` (Hessians): `.get<1>().get<1>()` extracts the second-order derivative.

4 Forward-Mode Jacobian

4.1 Mathematical basis

For $f : \mathbb{R}^n \to \mathbb{R}$, the j-th column $\partial f / \partial x_j$ of the Jacobian is obtained by seeding the j-th tangent to 1 and all others to 0:

$$f((x_1, 0), \ldots, (x_j, 1), \ldots, (x_n, 0)) = \left(f(\mathbf{x}),\ \frac{\partial f}{\partial x_j} \right).$$

Dual arithmetic propagates this tangent through every operation automatically. The full Jacobian requires n passes.

4.2 Code walkthrough: `detail::forward_mode_gradient`

Location: `gradient.hpp:58-91`.

```
template <ExpressionConcept Expr, ...>
auto forward_mode_gradient(Expr& expr, array<TArr, N> values)
    requires is_dual_v<typename Expr::value_type>
{
    using symbols    = extract_symbols_from_expr<Expr>::type;
    constexpr size_t n = mp::mp_size<symbols>::value;

    array<value_type, n>  seeds{};       // Dual seeds pushed into the tree
    array<scalar_type, n> gradients{};   // output

    [&]<size_t... Js>(index_sequence<Js...>) {
        // Step 1: init all to (value_j, 0)
        ((seeds[Js] = value_type{values[Js], scalar_type{}}), ...);

        // Step 2: one pass per variable j
        ([&]() {
            seeds[Js] = value_type{values[Js], scalar_type{1}}; // tangent = 1
            expr.update(symbols{}, seeds);                      // push into tree
            gradients[Js] = expr.eval().template get<1>();      // extract df/dxj
            seeds[Js] = value_type{values[Js], scalar_type{}};  // reset
        }(), ...);
    }(make_index_sequence<n>{});

    expr.update(symbols{}, seeds);   // Step 3: restore zero-tangent state
    return gradients;
}
```

Step by step:

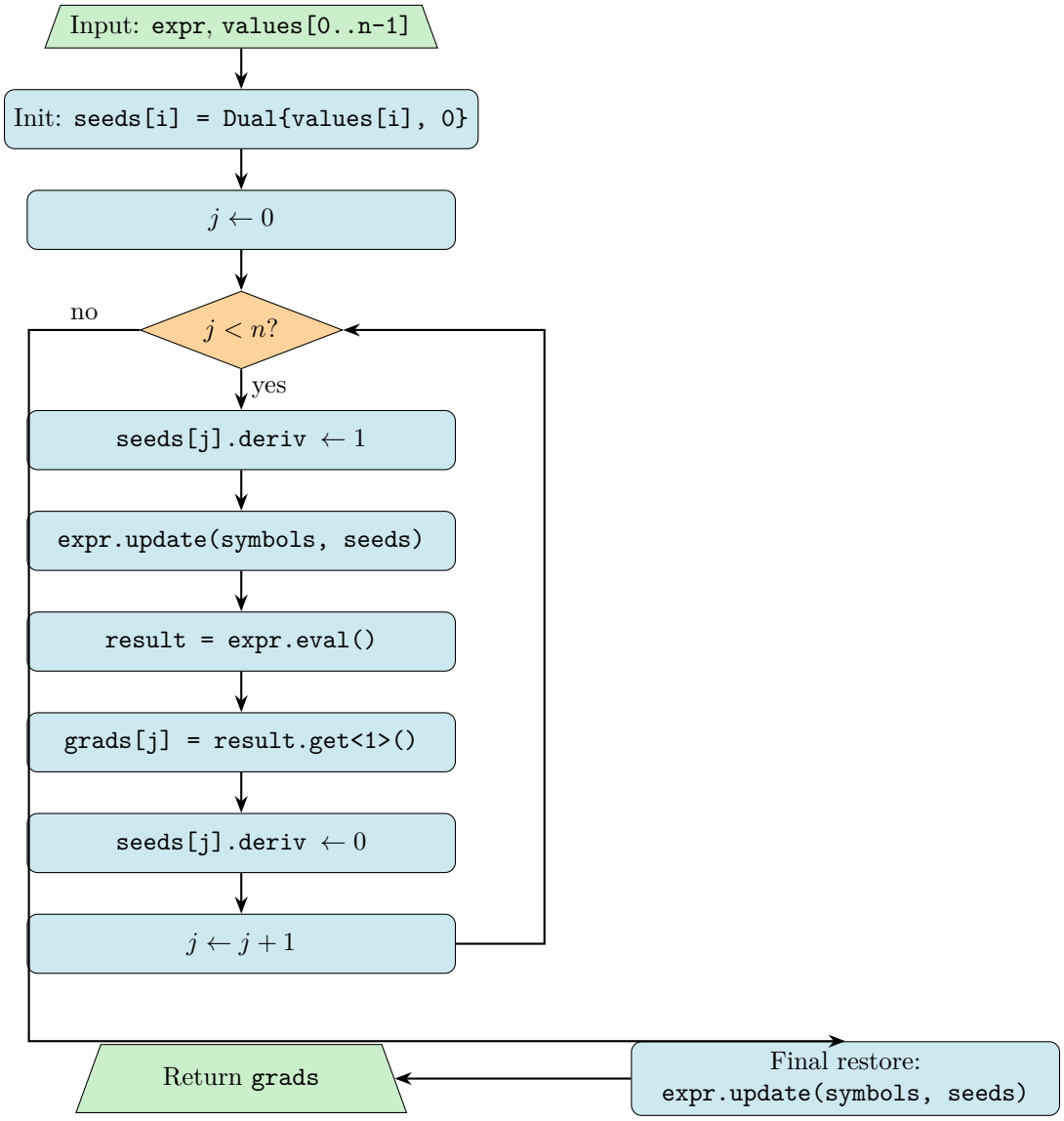
1. **Initialise seeds.** All seeds: `Dual{values[i], 0}`.
2. **One pass per j (compile-time loop unroll):**
   - Set `seeds[j].deriv = 1` (direction e_j).
   - `expr.update()` writes all seeds into the tree.
   - `expr.eval()` runs with dual arithmetic throughout. The returned `Dual` has `.get<1>()` $= \partial f / \partial x_j$.

- Store into `gradients[j]`, reset `seeds[j].deriv = 0`.

3. **Restore state.** Final `update` leaves the tree zero-tangent.

Complexity: $O(n \cdot |E|)$. Requires n full tree evaluations.

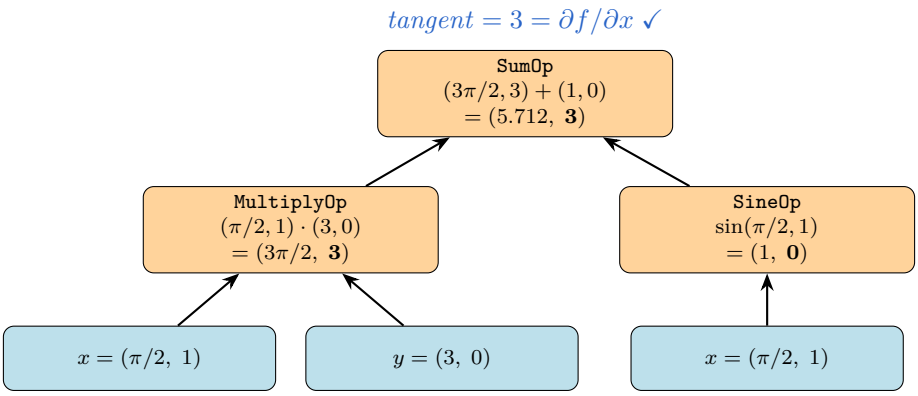
4.3 Flowchart



4.4 Worked example

Pass $j = 0$: compute $\partial f / \partial x$

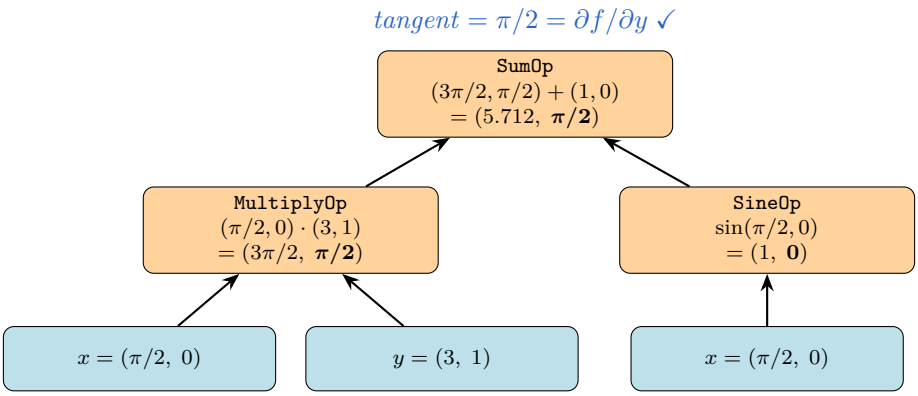
Seeds: $x = (\pi/2,\ 1), \quad y = (3,\ 0)$



Product rule: $\dot{(xy)} = \dot{x} \cdot y + x \cdot \dot{y} = 1 \cdot 3 + \frac{\pi}{2} \cdot 0 = 3$.
Chain rule on sine: $\dot{\sin x} = \cos(\pi/2) \cdot 1 = 0$.
Sum: $3 + 0 = \mathbf{3}$.

Pass $j = 1$: compute $\partial f / \partial y$

Seeds: $x = (\pi/2,\ 0), \quad y = (3,\ 1)$



Product rule: $\dot{(xy)} = 0 \cdot 3 + \frac{\pi}{2} \cdot 1 = \frac{\pi}{2}$.

5 Reverse-Mode Jacobian

5.1 Mathematical basis

Define the *adjoint* of node v as $\bar{v} = \partial f / \partial v$. Starting from the output ($\bar{f} = 1$), a single backward pass distributes adjoints to every input node via the chain rule. For $f = \ell(u, v)$:

$$\bar{u} \mathrel{+}= \bar{f} \cdot \frac{\partial \ell}{\partial u}, \qquad \bar{v} \mathrel{+}= \bar{f} \cdot \frac{\partial \ell}{\partial v}.$$

The $+=$ accumulation automatically handles variables appearing in multiple places (e.g. both x leaves contribute to the same $\bar{x}$).

5.2 Code walkthrough: `detail::reverse_mode_gradient`

Location: `gradient.hpp:31-40`.

```
template <ExpressionConcept Expr, typename T = typename Expr::value_type
    >
    requires (!is_dual_v<T>)
constexpr auto reverse_mode_gradient(const Expr& expr) {
```

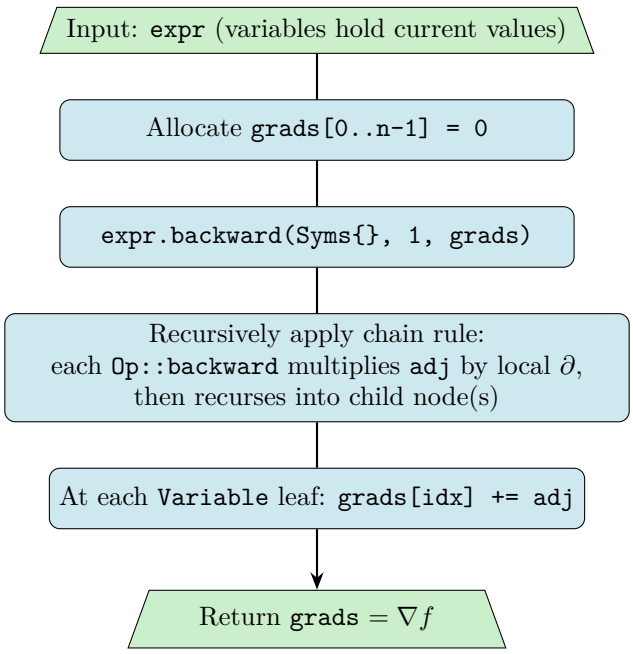
```
    using Syms = extract_symbols_from_expr<Expr>::type;
    constexpr auto N = mp::mp_size<Syms>::value;
    std::array<T, N> grads{};                // zero-init output
    expr.backward(Syms{}, T{1}, grads);      // seed root adjoint = 1
    return grads;
}
```

Step by step:

1. Allocate `grads[0..n-1] = 0`.
2. Call `expr.backward(Syms{}, 1, grads)`. Root adjoint $= 1$ because $\partial f/\partial f = 1$.
3. `backward` recursively applies the chain rule; at each `Variable` leaf, `grads[idx] += adj` accumulates the result.
4. Return `grads`: element i holds $\partial f/\partial x_i$.

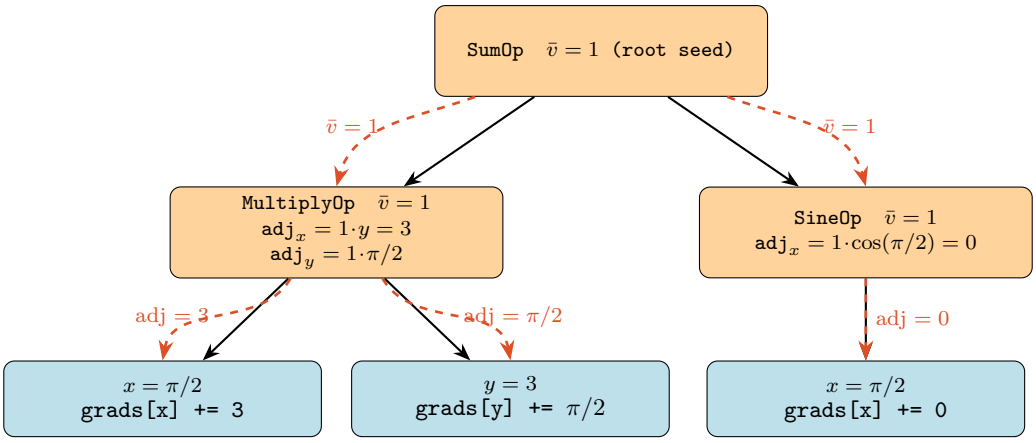
Complexity: $O(|E|)$ — a single traversal regardless of n. The expression is taken as `const Expr&`; the tree is not mutated.

5.3 Flowchart



5.4 Worked example

Variables already hold $x = \pi/2$, $y = 3$. The diagram shows adjoint flow (dashed orange) propagating top-down, while structural edges point bottom-up.



Accumulation at variable nodes:

$$\texttt{grads[x]} = 3 + 0 = 3 = y + \cos(x)\big|_{\pi/2,3} \checkmark$$
$$\texttt{grads[y]} = \tfrac{\pi}{2} = x \checkmark$$

Both x leaves contribute to the same `grads[idx_x]` slot via `+=` in `Variable::backward`. This is what makes reverse mode exact when variables appear multiple times in the expression (DAG, not just tree).

6 Comparison

Property	Forward mode	Reverse mode
Algorithm	Dual tangent propagation	Adjoint backpropagation
Passes required	n (one per input)	1
Tree mutation	Yes (`update` per pass)	No (`const Expr&`)
Output per pass	one $\partial f/\partial x_j$	full ∇f
Time complexity	$O(n \cdot \lvert E\rvert)$	$O(\lvert E\rvert)$
Preferred when	few inputs ($n \ll m$)	many inputs, scalar f
Variable type	`Variable<Dual<T>,'x'>`	`Variable<T,'x'>`

Table 3: Forward vs. reverse mode.

6.1 Public API (`gradient.hpp:196-211`)

```
// Reverse mode
auto g1 = gradient<diff::DiffMode::Reverse>(expr);

// Forward mode (expr must have Dual value_type)
auto g2 = gradient<diff::DiffMode::Forward>(expr, {x_val, y_val});

// Macros (gradient.hpp:256-257)
auto g3 = reverse_mode_grad(expr);
auto g4 = forward_mode_grad(expr, {x_val, y_val});
```

7 Symbol Resolution: Compile-Time Indexing

Every `Variable<T,symbol>` carries its name as a compile-time `char` parameter. All symbols in an expression are collected into a sorted, deduplicated Boost.MP11 type-list at compile time:

```
// traits.hpp
using Syms = typename extract_symbols_from_expr<Expr>::type;
// For f(x,y):
//   mp_list< integral_constant<char,'x'>,
//            integral_constant<char,'y'> >

// Index lookup -- zero runtime cost:
constexpr auto idx = find_index_of_char<'x', Syms>(); // == 0
constexpr auto idy = find_index_of_char<'y', Syms>(); // == 1
```

`update`, `collect`, `backward`, and `eval_seeded` all resolve their array slot via `find_index_of_char` at compile time — no runtime hash map, branch, or virtual dispatch.

8 Quick Reference

Forward mode: setup

```
// Variables must have Dual value type
Variable<Dual<double>,'x'> x{Dual{x_val, 0.0}}; // or PDV(x_val, 'x
    ')
Variable<Dual<double>,'y'> y{Dual{y_val, 0.0}};

auto f = x * y + sin(x);

auto grad = forward_mode_grad(f, {x_val, y_val});
// grad[0] = df/dx,  grad[1] = df/dy
```

Reverse mode: setup

```
// Variables hold plain values
Variable<double,'x'> x{x_val};  // or PV(x_val, 'x')
Variable<double,'y'> y{y_val};

auto f = x * y + sin(x);        // tree already holds x_val, y_val

auto grad = reverse_mode_grad(f);
// grad[0] = df/dx,  grad[1] = df/dy
```